



Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences

Finale Version der Arbeit

*Automatisierte Sicherheitstests in CI/CD-Pipelines:
DevSecOps als Ansatz für sichere
Softwareentwicklung*

Studiengang: Cyber Security & Privacy

Themenbereich: Software Engineering & Cybersecurity

Verfasser: Firas Massoussi

Modul: Literatur-Seminar

Prof. Dr. Hackelöer

Abgabe: Finale Version

Datum: 02.07.2026

1. Einleitung

1.1 Motivation und Kontext

Software wird heute in vielen Unternehmen sehr schnell entwickelt und regelmäßig verändert. Neue Funktionen werden oft nicht mehr erst nach langen Projektphasen veröffentlicht, sondern in kurzen Abständen gebaut, getestet und bereitgestellt. Dafür nutzen viele Teams agile Arbeitsweisen, DevOps und CI/CD-Pipelines. Diese Arbeitsweise kann Vorteile bringen, weil Fehler früher sichtbar werden und neue Funktionen schneller bei den Nutzenden ankommen.

Gleichzeitig entsteht dadurch ein Problem für die Sicherheit. Wenn Software sehr schnell entwickelt und veröffentlicht wird, darf Cybersecurity nicht erst am Ende geprüft werden. Sicherheitsprobleme wie unsichere Programmierung, bekannte Schwachstellen in Bibliotheken, falsch gesetzte Berechtigungen oder versehentlich veröffentlichte Zugangsdaten können sonst zu spät erkannt werden. Der Secure Software Development Framework von NIST beschreibt, dass sichere Entwicklungspraktiken in bestehende Entwicklungsprozesse eingebaut werden sollen und nicht nur als zusätzlicher Schritt am Ende verstanden werden dürfen (vgl. NIST, 2022).

Genau an dieser Stelle setzt DevSecOps an. DevSecOps bedeutet, dass Sicherheit als fester Teil von Entwicklung, Betrieb und automatisierten Abläufen verstanden wird. Die Open Worldwide Application Security Project (OWASP) Foundation ist eine gemeinnützige Organisation, die frei zugängliche Projekte, Standards und Hilfen zur Verbesserung der Softwaresicherheit bereitstellt (vgl. OWASP Foundation, o. J.). OWASP beschreibt DevSecOps als Ansatz, um sichere Pipelines aufzubauen und eine Shift-Left-Security-Kultur zu fördern (vgl. OWASP, o. J.-d).

Der Schwerpunkt dieser Arbeit liegt auf automatisierten Sicherheitstests in CI/CD-Pipelines. Solche Tests können zum Beispiel Quellcode, externe Bibliotheken, Zugangsdaten, Container-Images oder eine laufende Anwendung prüfen. Dadurch können Schwachstellen früher sichtbar werden. Aus der Verbindung von schneller Softwareauslieferung und früher Sicherheitsprüfung ergibt sich der konkrete Untersuchungsbedarf dieser Arbeit.

Vor diesem Hintergrund verbindet das Thema die Bereiche Software Engineering und Cybersecurity. Gerade in der Praxis arbeiten Entwicklungsteams häufig mit automatisierten Pipelines. Deshalb wird untersucht, wie Sicherheitstests in diese Abläufe eingebaut werden können, an welchen Stellen sie den größten Nutzen bieten und welche Grenzen dabei entstehen.

1.2 Problemstellung

In vielen Softwareprojekten wird Sicherheit noch zu spät geprüft. Häufig finden Sicherheitsprüfungen erst kurz vor der Veröffentlichung oder sogar erst nach dem Deployment statt. Kumar und Goyal beschreiben, dass Sicherheit und Datenschutz bei hohem Zeitdruck teilweise als aufwendig wahrgenommen und deshalb nachrangig behandelt werden. Ihr Modell

setzt dem kontinuierliche Sicherheitskontrollen im gesamten Ablauf entgegen (vgl. Kumar und Goyal, 2020). Auch NIST empfiehlt, sichere Entwicklungspraktiken in den bestehenden Softwareentwicklungsprozess zu integrieren, statt sie nur als Endkontrolle zu behandeln (vgl. NIST, 2022).

Ein weiteres Problem ist die starke Abhängigkeit moderner Software von externen Komponenten. Anwendungen verwenden Open-Source-Bibliotheken, Frameworks, Container-Images, Build-Systeme und Cloud-Dienste. Dadurch entstehen zusätzliche Angriffsflächen in der Software-Lieferkette. OWASP nennt unter anderem kompromittierte Abhängigkeiten, unsichere Build-Prozesse und Angriffe auf CI/CD-Systeme als Risiken (vgl. OWASP, o. J.-I). Akbar et al. zeigen zudem, dass fehlende sichere Entwicklungsstandards und fehlende automatisierte Sicherheitstests zu den besonders wichtigen Herausforderungen bei der Einführung von DevSecOps gehören (vgl. Akbar et al., 2022).

Zusätzlich können geheime Zugangsdaten versehentlich in Repositories landen. Dazu gehören zum Beispiel API-Keys, Tokens, Passwörter oder private Schlüssel. GitHub beschreibt Secret Scanning als Verfahren, das die Git-Historie und Repositories auf hartcodierte Zugangsdaten wie API-Keys, Passwörter und Tokens prüfen kann (vgl. GitHub Docs, o. J.). Solche Fehler sind besonders kritisch, weil Angreifer damit unter Umständen Zugriff auf Systeme oder Daten bekommen können.

Daraus ergibt sich die zentrale Problemstellung dieser Arbeit: Sicherheit muss in schnellen Entwicklungsprozessen früher, regelmäßiger und möglichst automatisiert berücksichtigt werden.

Die Forschungsfrage lautet: Wie können automatisierte Sicherheitstests in CI/CD-Pipelines dazu beitragen, Sicherheitsprobleme frühzeitig im Softwareentwicklungsprozess zu erkennen, und welche Vorteile sowie Herausforderungen ergeben sich daraus für das Software Engineering?

1.3 Zielsetzung

Ziel der Arbeit ist es, den Ansatz DevSecOps verständlich zu erklären und zu zeigen, wie automatisierte Sicherheitstests in CI/CD-Pipelines eingesetzt werden können. Dabei soll nicht nur beschrieben werden, welche Testarten existieren, sondern auch, an welcher Stelle im Entwicklungsprozess sie sinnvoll eingesetzt werden können.

Die Arbeit soll außerdem zeigen, wie sich DevSecOps vom klassischen DevOps unterscheidet. DevOps verbindet Entwicklung und Betrieb. DevSecOps ergänzt diesen Ansatz um Sicherheit. Dabei geht es nicht nur darum, ein weiteres Tool in eine Pipeline einzubauen. Wichtiger ist eine Arbeitsweise, bei der Sicherheit früh und regelmäßig mitgedacht wird. Diese Sichtweise passt zu der Literatur, die DevSecOps nicht nur als technische Lösung, sondern auch als organisatorischen Ansatz beschreibt (vgl. Rajapakse et al., 2022a; Zhao et al., 2024).

Ein weiteres Ziel ist der Vergleich verschiedener automatisierter Sicherheitstestarten. Dazu gehören SAST, DAST, Dependency Scanning beziehungsweise Software Composition

Analysis, Secret Scanning und Container Scanning. Die Arbeit soll zeigen, welche Art von Problem mit welcher Testart erkannt werden kann und warum ein einzelner Test nicht ausreicht.

Am Ende der Arbeit soll bewertet werden, welchen Nutzen DevSecOps für sichere Softwareentwicklung haben kann und welche Schwierigkeiten bei der Einführung entstehen. Vorteile können zum Beispiel frühere Fehlererkennung, regelmäßige Prüfungen und bessere Zusammenarbeit zwischen Entwicklung, Betrieb und Security sein. Herausforderungen können Fehlalarme, zusätzlicher Aufwand, schwierige Tool-Auswahl und fehlendes Security-Wissen im Team sein.

1.4 Aufbau des Papers

Die Arbeit ist in fünf Kapitel gegliedert. Kapitel 1 führt in das Thema ein, beschreibt Motivation und Problemstellung, formuliert die Forschungsfrage und erklärt die Zielsetzung der Arbeit.

Kapitel 2 erklärt die wichtigsten Grundlagen. Dazu gehören Software Engineering, sichere Softwareentwicklung, DevOps, DevSecOps, CI/CD-Pipelines und automatisierte Sicherheitstests. Außerdem werden verwandte Arbeiten eingeordnet, damit deutlich wird, welche Aspekte in der Literatur bereits behandelt werden.

Kapitel 3 beschreibt die Methodik. Dort wird erklärt, wie die Literaturrecherche durchgeführt wurde, welche Datenbanken und Suchbegriffe genutzt wurden und nach welchen Kriterien Quellen ausgewählt oder ausgeschlossen wurden.

Kapitel 4 stellt die Ergebnisse dar. Dort werden die Testarten in die Pipeline eingeordnet, anhand fester Kriterien verglichen und anschließend hinsichtlich Vorteilen und Herausforderungen bewertet. Kapitel 5 fasst die Ergebnisse zusammen, beantwortet die Forschungsfrage und nennt Grenzen der Arbeit.

2. Grundlagen und verwandte Arbeiten

2.1 Software Engineering und sichere Softwareentwicklung

Software Engineering beschäftigt sich mit der planvollen Entwicklung von Software. Dabei geht es nicht nur darum, Code zu schreiben. Wichtig sind auch Anforderungen, Entwurf, Tests, Wartung, Qualität und die Weiterentwicklung eines Systems. Sicherheit ist dabei ein wichtiger Teil der Qualität. Eine Anwendung kann fachlich funktionieren und trotzdem ein Risiko darstellen, wenn sie unsicher entwickelt wurde.

Sichere Softwareentwicklung bedeutet, dass Sicherheitsaspekte schon während der Entwicklung berücksichtigt werden. Dazu gehören zum Beispiel sichere Programmierregeln, Prüfung von Eingaben, Schutz von Zugangsdaten, sichere Konfigurationen und regelmäßige Sicherheitsprüfungen. NIST beschreibt im SSDF grundlegende sichere Entwicklungspraktiken,

die in unterschiedliche Softwareentwicklungsprozesse integriert werden können (vgl. NIST, 2022).

Für diese Arbeit ist wichtig, dass Sicherheit nicht als einmalige Abschlussprüfung verstanden wird. Sicherheit soll Schritt für Schritt im Entwicklungsprozess auftauchen. Automatisierte Sicherheitstests passen dazu, weil sie regelmäßig ausgeführt werden können. Dadurch wird Sicherheit eher zu einem wiederholbaren Bestandteil des Software Engineering und weniger zu einer Einzelprüfung am Ende.

Trotzdem darf sichere Softwareentwicklung nicht nur auf Tools reduziert werden. Tools können Hinweise liefern, aber Menschen müssen die Ergebnisse bewerten. Ein Tool kann zum Beispiel eine Warnung ausgeben, aber das Team muss entscheiden, ob es sich wirklich um ein kritisches Problem handelt und wie es behoben werden soll.

Für die praktische Umsetzung liefern OWASP ASVS und OWASP SAMM zusätzliche Orientierung. ASVS beschreibt überprüfbare Sicherheitsanforderungen für Anwendungen, während SAMM Organisationen dabei unterstützt, ihren sicheren Entwicklungsprozess schrittweise weiterzuentwickeln (vgl. OWASP ASVS, o. J.; OWASP SAMM, o. J.).

2.2 DevOps

DevOps ist ein Ansatz, der Entwicklung und Betrieb näher zusammenbringt. In klassischen Projekten sind diese Bereiche oft getrennt. Entwicklerinnen und Entwickler erstellen die Software, während der Betrieb später für Bereitstellung, Betrieb und Wartung zuständig ist. Diese Trennung kann zu langsamen Übergaben, Missverständnissen und Fehlern führen.

DevOps versucht, diese Trennung zu verringern. Teams sollen enger zusammenarbeiten und Abläufe stärker automatisieren. Typische Bestandteile sind Versionsverwaltung, automatisierte Builds, automatisierte Tests und CI/CD-Pipelines. Dadurch können Änderungen schneller geprüft und bereitgestellt werden.

Für die Sicherheit entsteht dadurch aber eine neue Herausforderung. Wenn Software häufiger veröffentlicht wird, müssen auch Sicherheitsprüfungen häufiger und schneller stattfinden. Rein manuelle Sicherheitsprüfungen können mit dem Tempo von DevOps oft schwer mithalten. Deshalb wird DevOps in vielen Quellen durch DevSecOps erweitert (vgl. Myrbakken und Colomo-Palacios, 2017).

2.3 DevSecOps und Shift Left Security

DevSecOps steht für Development, Security und Operations. Der Begriff beschreibt eine Erweiterung von DevOps um den Sicherheitsaspekt. Sicherheit soll nicht erst nach der Entwicklung geprüft werden, sondern von Anfang an Teil des Entwicklungsprozesses sein. Dieser Gedanke wird häufig mit Shift Left Security verbunden. Gemeint ist, dass Sicherheitsprüfungen früher im Prozess stattfinden sollen.

Myrbakken und Colomo-Palacios beschreiben DevSecOps als Reaktion auf die Geschwindigkeit von DevOps. Traditionelle Sicherheitsmethoden passen oft nicht gut zu schnellen DevOps-Prozessen, weil sie zu spät oder zu langsam eingesetzt werden (vgl. Myrbakken und Colomo-Palacios, 2017). Neuere Literatur zeigt außerdem, dass DevSecOps mehrere Dimensionen umfasst, zum Beispiel Definitionen, Herausforderungen, Praktiken, Tools und Metriken (vgl. Zhao et al., 2024).

Rajapakse et al. zeigen in einer systematischen Übersicht, dass DevSecOps sowohl technische als auch organisatorische Herausforderungen hat. Sie ordnen Herausforderungen unter anderem den Bereichen Menschen, Praktiken, Tools und Infrastruktur zu (vgl. Rajapakse et al., 2022a). Das ist für diese Arbeit wichtig, weil automatisierte Sicherheitstests nur ein Teil von DevSecOps sind.

OWASP beschreibt DevSecOps als Ansatz, um sichere Pipelines zu entwickeln, passende Sicherheitswerkzeuge einzusetzen und Shift-Left-Security zu unterstützen (vgl. OWASP, o. J.-d). Für diese Arbeit bedeutet DevSecOps daher vor allem: Sicherheit wird automatisiert und wiederholbar in den CI/CD-Prozess eingebaut. Trotzdem ersetzt Automatisierung nicht das Denken und Bewerten durch Menschen.

2.4 CI/CD-Pipelines

CI/CD steht für Continuous Integration und Continuous Delivery beziehungsweise Continuous Deployment. Bei Continuous Integration werden Codeänderungen regelmäßig zusammengeführt und automatisch geprüft. Bei Continuous Delivery oder Deployment wird Software nach erfolgreichen Prüfungen vorbereitet oder direkt in eine Umgebung ausgeliefert.

Eine einfache Pipeline kann zum Beispiel aus Commit, Build, Tests, Security-Scans, Image-Freigabe und Deployment bestehen. In jeden dieser Schritte können Prüfungen eingebaut werden. Funktionale Tests prüfen, ob die Software fachlich funktioniert. Sicherheitstests prüfen zusätzlich, ob sicherheitsrelevante Probleme vorhanden sind. Dadurch eignen sich CI/CD-Pipelines besonders gut, um Sicherheitstests regelmäßig und nachvollziehbar auszuführen.

Vereinfachte DevSecOps-Pipeline mit typischen automatisierten Sicherheitstests



Eigene Darstellung auf Basis der in der Arbeit betrachteten Testarten und Quellen.

Abbildung 1: Vereinfachte DevSecOps-Pipeline mit typischen automatisierten Sicherheitstests.
Eigene Darstellung auf Basis der betrachteten Quellen.

Die Pipeline selbst muss ebenfalls geschützt werden. Sie hat häufig Zugriff auf Quellcode, Secrets, Build-Artefakte und Deployment-Systeme. OWASP weist darauf hin, dass CI/CD-

Pipelines wegen ihrer Rolle im Software Development Life Cycle ein attraktives Ziel für Angriffe sein können und daher nicht ignoriert werden dürfen (vgl. OWASP, o. J.-b). Der OWASP Top 10 CI/CD Security Risks macht diesen Punkt ebenfalls deutlich, weil dort typische Risiken wie unsichere Pipeline-Konfigurationen, unzureichende Zugriffskontrolle und der Umgang mit Secrets als zentrale Themen genannt werden (vgl. OWASP, o. J.-c).

Auch die Software Supply Chain spielt hier eine Rolle. OWASP nennt als mögliche Bedrohungen unter anderem Dependency Confusion, manipulierte Build-Prozesse und unsichere Artefaktverwaltung; das Projekt „Top 10 CI/CD Security Risks“ ergänzt diese Sicht um typische Schwächen wie unsichere Pipeline-Konfigurationen oder zu weitgehende Berechtigungen (vgl. OWASP, o. J.-l; OWASP, o. J.-c). OpenSSF/SLSA bietet dazu ein Reifegradmodell, mit dem sich die Integrität von Build-Prozessen und Artefakten systematisch absichern lässt (vgl. OpenSSF/SLSA, o. J.).

2.5 Automatisierte Sicherheitstests

Automatisierte Sicherheitstests sind Prüfungen, die mit Werkzeugen automatisch ausgeführt werden. Sie können Quellcode, Abhängigkeiten, Container-Images oder eine laufende Anwendung prüfen. Der Vorteil besteht darin, dass solche Tests regelmäßig durchgeführt werden können. Dadurch werden Probleme früher sichtbar als bei einer einzelnen manuellen Prüfung am Ende.

Automatisierte Tests haben aber Grenzen. Sie finden nicht alle Schwachstellen und können Fehllalarme erzeugen. Deshalb müssen Ergebnisse bewertet und priorisiert werden. Ein Team muss entscheiden, welche Meldungen kritisch sind, welche nur dokumentiert werden und wann ein Build gestoppt wird. DevSecOps bedeutet deshalb nicht nur Tool-Einsatz, sondern auch klare Regeln und Verantwortlichkeiten.

2.5.1 SAST

SAST bedeutet Static Application Security Testing. Dabei wird Quellcode oder ein anderer statischer Softwarebestandteil geprüft, ohne dass die Anwendung laufen muss. OWASP beschreibt Static Code Analysis als Analyse von nicht laufendem Quellcode, bei der Werkzeuge mögliche Schwachstellen und unsichere Muster erkennen sollen (vgl. OWASP, o. J.-m).

SAST eignet sich besonders früh im Entwicklungsprozess, zum Beispiel beim Commit, im Pull Request oder während des Builds. Der Vorteil ist, dass Entwicklerinnen und Entwickler Probleme direkt im Code sehen können. Ein Nachteil ist, dass SAST nicht immer erkennt, wie sich die Anwendung zur Laufzeit verhält. Außerdem können Fehllalarme entstehen, die geprüft werden müssen.

2.5.2 DAST

DAST bedeutet Dynamic Application Security Testing. Dabei wird eine laufende Anwendung getestet. Das Tool prüft die Anwendung von außen, ähnlich wie ein Angreifer bestimmte

Anfragen oder Eingaben testen würde. DAST wird deshalb eher nach dem Build oder in einer Testumgebung eingesetzt.

Ein bekanntes Werkzeug ist OWASP ZAP. Die ZAP-Dokumentation beschreibt ZAP als Werkzeug, das für Security Testing von Webanwendungen eingesetzt werden kann und auch Möglichkeiten zur Automatisierung bietet (vgl. OWASP ZAP, o. J.). Der Vorteil von DAST ist, dass die laufende Anwendung betrachtet wird. Der Nachteil ist, dass der Test später in der Pipeline stattfindet und teilweise länger dauern kann.

2.5.3 Dependency Scanning und Software Composition Analysis

Dependency Scanning oder Software Composition Analysis prüft externe Bibliotheken und Komponenten. Das ist wichtig, weil moderne Anwendungen viele fremde Pakete verwenden. Eine Anwendung kann also auch dann unsicher sein, wenn der eigene Code sauber geschrieben ist, aber eine verwendete Bibliothek eine bekannte Schwachstelle enthält.

OWASP Dependency-Check ist ein Beispiel für ein SCA-Werkzeug. OWASP beschreibt es als Werkzeug, das Projektabhängigkeiten prüft und versucht, bekannte veröffentlichte Schwachstellen in diesen Abhängigkeiten zu erkennen (vgl. OWASP, o. J.-e). Solche Prüfungen passen gut in die Build-Phase, weil dort die verwendeten Abhängigkeiten sichtbar sind.

2.5.4 Secret Scanning

Secret Scanning prüft, ob Zugangsdaten im Quellcode, in Konfigurationsdateien oder in der Repository-Historie enthalten sind. Dazu gehören zum Beispiel API-Keys, Tokens, Passwörter oder private Schlüssel. Solche Daten sollten nicht im Code gespeichert werden, weil sie bei Veröffentlichung missbraucht werden können.

GitHub beschreibt Secret Scanning als Prüfung der Repository-Historie auf hartcodierte Zugangsdaten wie API-Keys, Passwörter und Tokens (vgl. GitHub Docs, o. J.). Secret Scanning sollte möglichst früh stattfinden, zum Beispiel beim Commit oder Pull Request. Wird ein Secret erst spät gefunden, muss es oft zurückgezogen und neu erstellt werden. Deshalb reicht es nicht, nur zu scannen; es braucht zusätzlich ein gutes Secrets Management. OWASP betont beim Secrets Management unter anderem die zentrale Verwaltung, Rotation und Kontrolle von Geheimnissen (vgl. OWASP, o. J.-j).

2.5.5 Container Scanning

Container Scanning prüft Container-Images auf bekannte Schwachstellen und unsichere Bestandteile. Das ist wichtig, wenn Anwendungen mit Docker oder ähnlichen Technologien ausgeliefert werden. Ein Container enthält nicht nur den eigenen Anwendungscode, sondern auch Pakete und Systembestandteile, die ebenfalls Schwachstellen enthalten können.

NIST beschreibt Container als portable, wiederverwendbare und automatisierbare Möglichkeit, Anwendungen zu verpacken und auszuführen, weist aber auch auf Sicherheitsrisiken und notwendige Gegenmaßnahmen hin (vgl. NIST, 2017). Container Scanning passt daher

besonders gut in die Phase vor dem Deployment. Wenn ein Image kritische Schwachstellen enthält, sollte es nicht ungeprüft veröffentlicht werden.

Zusammenfassend lassen sich die Testarten nach ihrem Prüfgegenstand und ihrem typischen Einsatzzeitpunkt einordnen. SAST untersucht den Quellcode bereits beim Commit, Pull Request oder Build. DAST benötigt dagegen eine laufende Anwendung und wird deshalb meist in einer Testumgebung eingesetzt. Dependency Scanning prüft externe Bibliotheken während des Builds, Secret Scanning sucht möglichst früh im Repository nach Zugangsdaten und Container Scanning kontrolliert das fertige Image vor dem Deployment. Die genaue Bewertung ihrer Stärken, Grenzen und ihres Zusammenspiels erfolgt im Ergebniskapitel.

2.6 Verwandte Arbeiten und Einordnung

Die betrachtete Forschung zeigt, dass DevSecOps nicht nur ein technisches Thema ist. Myrbakken und Colomo-Palacios ordnen DevSecOps als Verbindung von DevOps und Sicherheitspraktiken ein (vgl. Myrbakken und Colomo-Palacios, 2017). Zhao et al. strukturieren das Forschungsfeld in Definitionen, Herausforderungen, Praktiken, Werkzeuge und Metriken und führen diese Bereiche in einem Lebenszyklusmodell zusammen (vgl. Zhao et al., 2024). Prates und Pereira identifizieren darüber hinaus 39 DevSecOps-Praktiken und ordnen unter anderem SAST, DAST, automatisierte Sicherheitstests, Continuous Monitoring, Threat Modeling und Zusammenarbeit den Kernpraktiken zu (vgl. Prates und Pereira, 2025).

Weitere Studien ergänzen diese übergeordnete Sicht. Rajapakse et al. ordnen Herausforderungen und Lösungen den Bereichen Menschen, Praktiken, Werkzeuge und Infrastruktur zu (vgl. Rajapakse et al., 2022a). Ihre empirische Untersuchung zur Zusammenarbeit bei Application Security Testing zeigt außerdem, dass unklare Rollen, fehlende gemeinsame Ziele und begrenzte Unterstützung durch Werkzeuge die Zusammenarbeit erschweren können (vgl. Rajapakse et al., 2022b). Kumar und Goyal schlagen mit ADOC ein Modell vor, das Sicherheitskontrollen an mehreren Kontrollpunkten eines automatisierten Workflows verankert (vgl. Kumar und Goyal, 2020). Feio et al. leiten aus Literatur und Fallstudie konkrete Security-Aktivitäten für die Phasen Plan, Code, Build, Test, Release, Deploy, Operate und Monitor ab (vgl. Feio et al., 2024).

Die Praxisquellen von OWASP und NIST ergänzen diese wissenschaftlichen Arbeiten. OWASP liefert konkrete Hinweise zu DevSecOps, CI/CD-Sicherheit, statischer Codeanalyse, Dependency-Checks und sicheren Anforderungen. NIST liefert mit dem SSDF einen Rahmen für sichere Softwareentwicklung und mit SP 800-190 Hinweise zur Containersicherheit. Für diese Arbeit ist die Kombination aus wissenschaftlichen und fachlichen Quellen sinnvoll, weil DevSecOps sowohl ein Forschungsthema als auch ein praktisches Thema ist.

Damit ergibt sich für die vorliegende Arbeit ein klarer Fokus: Es geht nicht nur darum, einzelne Tools aufzuzählen. Wichtiger ist die Frage, wie die verschiedenen Testarten gemeinsam in einer Pipeline wirken und welche Vorteile und Grenzen diese Arbeitsweise hat.

3. Methodik

3.1 Art der Arbeit

Die Arbeit wurde als Literaturarbeit erstellt. Die Forschungsfrage wurde auf Grundlage vorhandener wissenschaftlicher und fachlicher Quellen beantwortet. Es wurde keine eigene Umfrage und kein eigenes technisches Experiment durchgeführt. Stattdessen wurden geeignete Quellen gesucht, gelesen, verglichen und zusammengeführt.

Diese Vorgehensweise passt zum Thema, weil DevSecOps bereits in vielen wissenschaftlichen und fachlichen Quellen behandelt wird. Außerdem geht es in dieser Arbeit nicht darum, ein bestimmtes Tool praktisch zu messen, sondern darum, den Einsatz automatisierter Sicherheitstests in CI/CD-Pipelines einzuordnen und zu bewerten.

3.2 Recherchequellen und Datenbanken

Für die Literaturrecherche wurden wissenschaftliche Datenbanken und ergänzende fachliche Quellen kombiniert. Wissenschaftliche Beiträge wurden vor allem über ACM Digital Library, IEEE Xplore, SpringerLink, ScienceDirect und Google Scholar gesucht. Neben Überblicksarbeiten wurden auch konzeptionelle Studien, empirische Untersuchungen und Fallstudien aufgenommen, damit der Ergebnisteil nicht nur Begriffe erklärt, sondern konkrete Pipeline-Erweiterungen und organisatorische Folgen einordnen kann.

Ergänzend wurden anerkannte Praxisquellen verwendet, vor allem OWASP, NIST, GitHub Docs, OWASP ZAP und OpenSSF/SLSA. Diese Quellen ersetzen die wissenschaftliche Literatur nicht. Sie werden dort eingesetzt, wo konkrete Standards, Testarten, Sicherheitsanforderungen oder technische Hinweise beschrieben werden. In der Auswertung wird daher klar zwischen peer-reviewter Forschung und fachlichen Praxisquellen unterschieden.

3.3 Suchbegriffe und Suchstrings

Die Recherche begann mit allgemeinen Suchbegriffen. Dazu gehörten DevSecOps, DevOps Security, CI/CD Security, Secure Software Development und Automated Security Testing. Danach wurden genauere Begriffe verwendet, zum Beispiel Static Application Security Testing, Dynamic Application Security Testing, Software Composition Analysis, Dependency Scanning, Secret Scanning, Container Scanning und Shift Left Security.

Beispielhafte Suchstrings waren: DevSecOps AND Software Engineering, DevSecOps AND CI/CD, Security Testing AND CI/CD Pipeline, Secure Software Development AND DevSecOps, Automated Security Testing AND DevOps sowie Shift Left Security AND DevOps. Diese Suchstrings wurden je nach Datenbank angepasst, weil nicht jede Datenbank dieselben Suchfunktionen verwendet.

Bei der Suche wurde außerdem darauf geachtet, dass die Quellen einen klaren Bezug zur Forschungsfrage haben. Quellen waren besonders relevant, wenn sie erklären, wie Sicherheit in DevOps oder CI/CD integriert werden kann, welche Testarten dafür geeignet sind oder welche Herausforderungen bei der Einführung entstehen.

3.4 Ein- und Ausschlusskriterien

Bei der Auswahl der Quellen wurde zuerst der Titel geprüft. Danach wurde das Abstract gelesen. Wenn eine Quelle zum Thema passte, wurde der Text genauer betrachtet. Wichtig war ein klarer Bezug zu DevSecOps, CI/CD-Pipelines, automatisierten Sicherheitstests oder sicherer Softwareentwicklung.

Eingeschlossen wurden wissenschaftliche Quellen wie Konferenzbeiträge, Zeitschriftenartikel und Bücher. Fachliche Quellen wurden genutzt, wenn sie von anerkannten Organisationen stammen oder direkt für das Thema wichtig sind. Dazu gehören zum Beispiel OWASP, NIST, OpenSSF und offizielle Tool-Dokumentationen.

Ausgeschlossen wurden Quellen ohne klaren Bezug zur Forschungsfrage. Reine Werbetexte, unklare Blogbeiträge und Quellen ohne erkennbare Autorenschaft wurden nicht als Hauptquellen verwendet. Blogbeiträge dienten höchstens als Einstieg, aber nicht als zentrale Grundlage der Arbeit.

3.5 Methode zur Beantwortung der Forschungsfrage

Die Forschungsfrage wurde in mehrere Teilfragen aufgeteilt. Zuerst wurde geklärt, was DevSecOps bedeutet und wie es sich von DevOps unterscheidet. Danach wurde untersucht, welche Rolle CI/CD-Pipelines für moderne Softwareentwicklung spielen. Anschließend wurden verschiedene Sicherheitstestarten beschrieben und miteinander verglichen.

Der Vergleich erfolgte anhand fester Kriterien. Betrachtet wurden der Einsatzzeitpunkt in der Pipeline, das geprüfte Objekt, typische Sicherheitsprobleme, Stärken und Grenzen. Dadurch wurde verhindert, dass die Arbeit nur Toolnamen aufzählt. Stattdessen wurde sichtbar, welche Testart welchen Beitrag zu einer sicheren Pipeline leisten kann.

Danach wurden Vorteile und Herausforderungen aus der Literatur gesammelt und geordnet. Zu den Vorteilen gehören frühere Fehlererkennung, regelmäßige Prüfungen und bessere Zusammenarbeit. Zu den Herausforderungen gehören Fehlalarme, zusätzlicher Aufwand, fehlendes Sicherheitswissen und die Gefahr, sich zu stark auf Tools zu verlassen.

Die Quellen wurden nicht nur einzeln zusammengefasst, sondern miteinander verbunden. Wenn mehrere Quellen ähnliche Punkte nannten, wurde daraus ein gemeinsames Muster abgeleitet. Unterschiedliche Schwerpunkte wurden in der Auswertung berücksichtigt. Dadurch beschreibt die Arbeit nicht nur, sondern vergleicht und bewertet die Ergebnisse.

3.6 Grenzen der Methodik

Die Arbeit hat einige Grenzen. Da es sich um eine Literaturarbeit handelt, werden keine eigenen Messwerte aus einer echten CI/CD-Pipeline erhoben. Es wird also nicht praktisch getestet, wie schnell ein bestimmtes Tool arbeitet oder wie viele Schwachstellen es findet. Die Bewertung erfolgt auf Grundlage vorhandener Literatur und fachlicher Quellen.

Eine weitere Grenze ist, dass sich Tools und Best Practices im Bereich DevSecOps schnell verändern. Neue Tools entstehen, alte Tools ändern sich und Sicherheitsanforderungen entwickeln sich weiter. Deshalb wird in der Arbeit eher der grundsätzliche Einsatz von Testarten betrachtet und weniger ein einzelnes Tool als dauerhaft beste Lösung dargestellt.

Außerdem kann die Arbeit nicht alle möglichen Sicherheitstests vollständig behandeln. Der Schwerpunkt liegt auf SAST, DAST, Dependency Scanning, Secret Scanning und Container Scanning, weil diese Testarten besonders gut zu CI/CD-Pipelines passen und häufig im Zusammenhang mit DevSecOps genannt werden.

3.7 Hinweis zur Arbeitsweise und zu verwendeten Hilfsmitteln

Bei der Erstellung dieser Arbeit wurden digitale Hilfsmittel zur sprachlichen Überarbeitung, Strukturierung und Formatkontrolle genutzt. Die inhaltliche Auswahl der Quellen, die Bewertung der Aussagen und die Verantwortung für den Text liegen beim Verfasser. Alle übernommenen Gedanken aus fremden Quellen werden im Text durch Quellenverweise gekennzeichnet und im Literaturverzeichnis aufgeführt.

Dieser Hinweis dient der Transparenz. Die Arbeit soll dadurch nachvollziehbar machen, welche Rolle Hilfsmittel hatten: Sie unterstützen bei Formulierung und Ordnung des Textes, ersetzen aber nicht die eigene Prüfung der Quellen und die fachliche Auseinandersetzung mit dem Thema.

4. Ergebnisse und Bewertung

4.1 DevSecOps als Erweiterung von DevOps

Die Literatur zeigt übereinstimmend, dass DevSecOps DevOps nicht ersetzt, sondern dessen Grundidee erweitert. DevOps verbindet Entwicklung und Betrieb, automatisiert Abläufe und verkürzt den Weg von einer Codeänderung bis zur Bereitstellung. DevSecOps ergänzt diesen Ablauf um Sicherheitsanforderungen, automatisierte Kontrollen, gemeinsame Verantwortlichkeiten und Sicherheitsrückmeldungen über den gesamten Lebenszyklus (vgl. Myrbakken und Colomo-Palacios, 2017; Kumar und Goyal, 2020; Zhao et al., 2024).

Der Unterschied liegt deshalb nicht nur im zusätzlichen Begriff „Security“. Im klassischen DevOps kann Sicherheit weiterhin als eigener Prüfschritt vor einer Veröffentlichung behandelt werden. In DevSecOps werden dagegen bereits in der Planung Sicherheitsanforderungen und Bedrohungen betrachtet, während der Entwicklung Code und Secrets geprüft, im Build

Abhängigkeiten und Artefakte kontrolliert, in der Testphase laufende Anwendungen untersucht und im Betrieb Sicherheitsereignisse überwacht werden (vgl. Feio et al., 2024; Prates und Pereira, 2025).

Damit verändert DevSecOps drei Bereiche gleichzeitig: den Prozess, weil Security-Aktivitäten in die Pipeline aufgenommen werden; die Verantwortung, weil Entwicklung, Betrieb und Security gemeinsam handeln; und die Automatisierung, weil wiederholbare Kontrollen direkt in CI/CD ausgeführt werden.

Im direkten Vergleich verfolgt DevOps vor allem das Ziel, Software schnell und zuverlässig bereitzustellen. DevSecOps behält dieses Ziel bei, ergänzt es aber um eine laufende Risikoreduktion. Neben Development und Operations wird Security als gemeinsame Verantwortung einbezogen. Sicherheitsprüfungen finden dadurch nicht nur kurz vor einer Veröffentlichung statt, sondern bereits von der Planung bis zum Betrieb (vgl. Kumar und Goyal, 2020; Rajapakse et al., 2022a; Zhao et al., 2024).

Auch die Automatisierung wird erweitert. Während DevOps vor allem Build, funktionale Tests und Deployment automatisiert, ergänzt DevSecOps unter anderem Security-Scans, Security Gates und Policy-Prüfungen. Das Feedback umfasst damit nicht nur Funktion, Qualität und Verfügbarkeit, sondern zusätzlich Schwachstellen, Risiken und sicherheitsbezogene Ereignisse. Im Betrieb kommen Security Monitoring und Incident Response hinzu. Erfolg wird deshalb nicht nur an Geschwindigkeit und Stabilität gemessen, sondern auch daran, ob Risiken früh erkannt und angemessen behandelt werden (vgl. Prates und Pereira, 2025; Feio et al., 2024).



Kernaussage: DevSecOps ersetzt DevOps nicht, sondern erweitert dessen Prozess, Rollen und Automatisierung um kontinuierliche Sicherheit.

Abbildung 2: Gegenüberstellung von DevOps und DevSecOps. Eigene Darstellung auf Grundlage von Kumar und Goyal (2020), Rajapakse et al. (2022a), Zhao et al. (2024) und Prates und Pereira (2025).

4.2 Einordnung der Sicherheitstests in CI/CD-Pipelines

Eine DevSecOps-Pipeline erweitert den klassischen Ablauf an mehreren konkreten Stellen. In der Planungsphase werden Sicherheitsanforderungen, Risiken und Bedrohungsmodelle festgelegt. In der Code-Phase folgen sichere Programmierrichtlinien, Code Reviews, SAST und Secret Scanning. Während des Builds werden Abhängigkeiten mit SCA geprüft; zusätzlich können Infrastructure-as-Code-Dateien und erzeugte Artefakte kontrolliert werden (vgl. Kumar und Goyal, 2020; Feio et al., 2024).

In der Testphase wird die gebaute oder laufende Anwendung untersucht. Hier passen DAST, Fuzzing und Integrationstests. Vor der Freigabe können Security Gates festlegen, ob kritische Funde einen Release stoppen. Beim Deployment werden Container-Images, Konfigurationen und Berechtigungen geprüft. Im Betrieb und Monitoring kommen Patching, Protokollauswertung, Schwachstellenüberwachung und Incident Response hinzu (vgl. Feio et al., 2024; Prates und Pereira, 2025).

Diese Einordnung zeigt, dass die fünf in dieser Arbeit genauer betrachteten Testarten nur einen Teil des Gesamtprozesses bilden. Sie werden durch organisatorische Regeln und weitere technische Kontrollen ergänzt. Abbildung 1 stellt diese Erweiterung über alle acht Phasen dar. Entscheidend ist, dass nicht überall derselbe Test wiederholt wird, sondern jede Phase eine zu ihrem Prüfobjekt passende Sicherheitsaktivität erhält.

Feio et al. zeigen in einer Fallstudie, dass die Umstellung einer vorhandenen DevOps-Pipeline konkrete Anpassungen an Phasen, Aktivitäten und Werkzeugen erfordert. Die Entwickler bewerteten die eingesetzten Werkzeuge grundsätzlich positiv, gleichzeitig blieben Aufwand, Fehlalarme und fehlende Expertise wichtige Grenzen (vgl. Feio et al., 2024). Damit wird DevSecOps als praktische Prozessänderung greifbar und nicht nur als allgemeine Forderung nach „mehr Sicherheit“.

4.3 Vergleich der Testarten

Der Vergleich der Testarten erfolgt nach den in der Methodik genannten Kriterien: Pipeline-Phase, Prüfobjekt, typische Funde, Stärken und Grenzen. Diese Kriterien passen zu der Forschung, die DevSecOps-Praktiken nicht nur nach Werkzeugnamen, sondern nach ihrer Rolle im Lebenszyklus einordnet (vgl. Zhao et al., 2024; Prates und Pereira, 2025; Feio et al., 2024). Dadurch wird klarer, dass die Testarten unterschiedliche Aufgaben haben und sich eher ergänzen, statt sich gegenseitig zu ersetzen.

Tabelle 1: Vergleich automatisierter Sicherheitstestarten in CI/CD-Pipelines. Eigene Darstellung auf Grundlage von NIST (2022), OWASP (o. J.-m), OWASP ZAP (o. J.), OWASP (o. J.-e), GitHub Docs (o. J.) und Souppaya et al. (2017).

Testart	Typische Pipeline-Phase	Prüfobjekt	Typische Funde	Stärken	Grenzen
SAST	Commit, Pull Request, Build	Quellcode / statischer Code	Unsichere Muster, Codefehler, mögliche Schwachstellen	Sehr früh einsetzbar; Fundstelle im Code sichtbar	Erkennt Laufzeit- und Konfigurationsprobleme nur begrenzt; mögliche Fehlalarme
DAST	Testumgebung nach Build	Laufende Anwendung	Fehler im Verhalten der Anwendung, z. B. unsichere Eingaben	Prüft die Anwendung von außen im laufenden Zustand	Später in der Pipeline; kann mehr Zeit benötigen; Codeursache nicht immer direkt sichtbar
SCA / Dependency Scanning	Build / Dependency-Check	Externe Bibliotheken und Pakete	Bekannte CVEs in Abhängigkeiten	Wichtig für Software Supply Chain; gut automatisierbar	Abhängig von Datenbanken; Risiko muss trotzdem bewertet werden
Secret Scanning	Commit, Pull Request, Repository-Scan	Code, Konfiguration, Git-Historie	API-Keys, Tokens, Passwörter, private Schlüssel	Sehr frühe Erkennung kritischer Geheimnisse	Secrets müssen danach rotiert und sicher verwaltet werden
Container Scanning	Image / Release vor Deployment	Container-Image und enthaltene Pakete	Bekannte Schwachstellen in Image und Systempaketen	Wichtig vor Veröffentlichung von Images	Bewertet nicht automatisch Architektur oder Laufzeitkonfiguration

Aus der Tabelle wird deutlich, dass kein einzelner Test alle Risiken abdeckt. SAST ist stark bei frühem Codefeedback, DAST ergänzt diese Sicht durch Tests an der laufenden Anwendung. Dependency Scanning und Container Scanning sind besonders wichtig für die Lieferkette, während Secret Scanning ein sehr konkretes Risiko im Repository adressiert. Für eine sichere Pipeline ist deshalb eine Kombination mehrerer Prüfungen sinnvoll.

Gleichzeitig muss das Team festlegen, welche Funde kritisch sind und wann ein Build gestoppt wird. Sonst besteht die Gefahr, dass zu viele Meldungen entstehen und wichtige Warnungen übersehen werden. Dieser Punkt verbindet die technische Ebene mit der organisatorischen Ebene von DevSecOps.

4.4 Vorteile von DevSecOps

Ein wichtiger Vorteil von DevSecOps ist die frühere Erkennung von Sicherheitsproblemen. Wenn Prüfungen bereits beim Commit, Pull Request oder Build ausgeführt werden, erhält das Team näher am Entstehungsort eine Rückmeldung. Rajapakse et al. nennen Shift Left und kontinuierliche Sicherheitsbewertung als zentrale Praktiken; Prates und Pereira ordnen SAST, DAST, automatisierte Sicherheitstests und Continuous Monitoring den Kernpraktiken von DevSecOps zu (vgl. Rajapakse et al., 2022a; Prates und Pereira, 2025).

Ein weiterer Vorteil ist die Regelmäßigkeit. Manuelle Sicherheitsprüfungen finden häufig nur zu bestimmten Zeitpunkten statt. Automatisierte Tests können dagegen bei jeder Änderung oder regelmäßig in der Pipeline laufen. OWASP beschreibt DevSecOps als Ansatz, um Sicherheitswerkzeuge in Pipelines einzubinden und eine Shift-Left-Kultur zu fördern (vgl. OWASP, o. J.-d).

DevSecOps kann außerdem die Zusammenarbeit verbessern. Entwicklung, Betrieb und Security müssen gemeinsame Regeln definieren. Das kann helfen, Sicherheit nicht als externe Kontrolle zu sehen, sondern als gemeinsame Aufgabe. Rajapakse et al. zeigen, dass

Zusammenarbeit, klare Rollen und gemeinsame Ziele zentrale Punkte bei der Einführung von DevSecOps sind (vgl. Rajapakse et al., 2022a; Rajapakse et al., 2022b).

Ein weiterer Vorteil ist die bessere Nachvollziehbarkeit. Wenn Tests in der Pipeline laufen, entstehen Berichte und Ergebnisse. Diese können zeigen, welche Prüfungen durchgeführt wurden und welche Probleme gefunden wurden. Das kann auch für Qualitätssicherung, spätere Audits oder interne Dokumentation hilfreich sein.

4.5 Herausforderungen von DevSecOps

Trotz der Vorteile gibt es mehrere Herausforderungen. Eine große Herausforderung sind Fehllalarme und eine hohe Zahl an Meldungen. Wenn Ergebnisse nicht priorisiert werden, können wichtige Warnungen übersehen werden. In der Fallstudie von Feio et al. konnten Funde wegen fehlender Expertise und des hohen Volumens nicht vollständig bereinigt werden. Dies zeigt, dass Automatisierung ohne Triage und Fachwissen keine verlässliche Aussage über den tatsächlichen Sicherheitsstand liefert (vgl. Feio et al., 2024).

Eine weitere Herausforderung ist der Aufwand bei der Einführung. Eine bestehende Pipeline muss angepasst werden; Werkzeuge müssen ausgewählt, eingerichtet und gepflegt werden. Außerdem sind Regeln nötig, wann ein Fund einen Build stoppt. Akbar et al. priorisieren unter anderem fehlende sichere Coding-Standards, fehlende automatisierte Security-Tools und mangelndes Wissen über statische Sicherheitstests als besonders wichtige Herausforderungen (vgl. Akbar et al., 2022). Rajapakse et al. bestätigen, dass technische und organisatorische Probleme gemeinsam betrachtet werden müssen (vgl. Rajapakse et al., 2022a).

Auch fehlendes Security-Wissen kann ein Problem sein. Wenn ein Entwicklungsteam nicht versteht, was eine Meldung bedeutet, kann es schwierig sein, richtig zu reagieren. Automatisierte Tests liefern Hinweise, aber sie ersetzen keine fachliche Bewertung. Das passt auch zu OWASP Secure Code Review, weil automatisierte Tools dort eher als Unterstützung und nicht als Ersatz für menschliche Prüfung verstanden werden (vgl. OWASP, o. J.-k).

Ein weiteres Risiko besteht darin, sich zu stark auf Tools zu verlassen. DevSecOps bedeutet nicht, dass Sicherheit automatisch vollständig gelöst ist. Architekturentscheidungen, Rechtekonzepte, Bedrohungsmodelle und manuelle Reviews bleiben wichtig. Außerdem muss die CI/CD-Pipeline selbst geschützt werden, weil sie Zugriff auf Code, Secrets und Deployment-Systeme hat (vgl. OWASP, o. J.-b; OWASP, o. J.-c).

4.6 Übergeordnetes Ergebnis: DevSecOps als Gesamtsystem

Die Einzelergebnisse lassen sich zu einem übergeordneten Bild zusammenführen. DevSecOps ist ein sozio-technisches System, in dem vier Ebenen zusammenwirken: Menschen, Praktiken, Werkzeuge und Infrastruktur. Diese Einordnung lehnt sich an die systematische Übersicht von Rajapakse et al. an und wird durch das Lebenszyklusmodell von Zhao et al. ergänzt (vgl. Rajapakse et al., 2022a; Zhao et al., 2024).

Auf der Ebene der Menschen sind gemeinsame Verantwortung, klare Rollen und Security-Wissen entscheidend. Die Ebene der Praktiken umfasst unter anderem Shift Left, Threat Modeling, sichere Anforderungen, Security Gates und eine kontinuierliche Bewertung. Werkzeuge unterstützen diese Praktiken durch SAST, DAST, SCA, Secret Scanning, Container Scanning und Monitoring. Zur Infrastruktur gehören die CI/CD-Plattform, Build-Systeme, Secrets, Artefakte und Laufzeitumgebungen, die ebenfalls geschützt werden müssen (vgl. Kumar und Goyal, 2020; Prates und Pereira, 2025).

Das Zusammenspiel der vier Ebenen lässt sich folgendermaßen beschreiben: Im Mittelpunkt steht eine kontinuierliche und sichere Softwarelieferung von der Planung bis zum Monitoring. Die Menschen legen Verantwortlichkeiten fest, bringen Security-Wissen ein und bewerten die Ergebnisse der Tests. Die Praktiken geben vor, wie Sicherheit umgesetzt wird, zum Beispiel durch Shift Left, Threat Modeling, Security Gates und regelmäßige Bewertungen. Die Werkzeuge automatisieren konkrete Prüfungen wie SAST, DAST, SCA, Secret Scanning, Container Scanning und Monitoring. Die Infrastruktur stellt die technische Grundlage bereit und muss CI/CD-Systeme, Secrets, Artefakte und Laufzeitumgebungen schützen. Erst durch die Verbindung dieser vier Bereiche entsteht ein durchgängiger DevSecOps-Prozess. Ein einzelnes Tool reicht dafür nicht aus, weil Funde ohne klare Zuständigkeiten, passende Regeln und geschützte Systeme wirkungslos bleiben können.

Das zentrale Ergebnis der Literaturlauswertung lautet daher: Automatisierte Sicherheitstests schaffen vor allem dann einen Nutzen, wenn sie an geeigneten Kontrollpunkten der Pipeline ausgeführt, durch klare Prozesse unterstützt und in einen kontinuierlichen Feedbackkreislauf eingebettet werden. DevSecOps ist damit nicht nur eine technische Erweiterung von DevOps, sondern ein Gesamtsystem aus Verantwortung, Sicherheitspraktiken, automatisierten Werkzeugen und geschützter Infrastruktur.

5. Fazit

Die Forschungsfrage lautete: Wie können automatisierte Sicherheitstests in CI/CD-Pipelines dazu beitragen, Sicherheitsprobleme frühzeitig im Softwareentwicklungsprozess zu erkennen, und welche Vorteile sowie Herausforderungen ergeben sich daraus für das Software Engineering?

Die Arbeit kommt zu dem Ergebnis, dass automatisierte Sicherheitstests Sicherheitsprobleme früher sichtbar machen können, wenn sie sinnvoll in verschiedene Phasen der Pipeline eingebaut werden. SAST und Secret Scanning eignen sich besonders früh im Prozess, weil sie bereits beim Commit, Pull Request oder Build eingesetzt werden können. Dependency Scanning beziehungsweise SCA passt gut in die Build-Phase, weil dort Abhängigkeiten sichtbar sind. DAST prüft eine laufende Anwendung und eignet sich deshalb eher für eine Testumgebung. Container Scanning ist besonders vor dem Deployment sinnvoll, wenn das fertige Container-Image geprüft wird.

Die Vorteile von DevSecOps liegen vor allem in früherer Rückmeldung, regelmäßigen Prüfungen, besserer Nachvollziehbarkeit und stärkerer Zusammenarbeit zwischen Entwicklung, Betrieb und Security. Gleichzeitig zeigt die Arbeit, dass DevSecOps nicht automatisch alle Sicherheitsprobleme löst. Herausforderungen sind Fehlalarme, zusätzlicher Aufwand, Tool-Auswahl, fehlendes Security-Wissen und die Gefahr, sich zu stark auf automatisierte Tools zu verlassen.

Für das Software Engineering ist DevSecOps deshalb ein sinnvoller Ansatz, wenn es als Gesamtsystem verstanden wird. Die Literatur zeigt, dass technische Kontrollen, sichere Praktiken, geschützte Infrastruktur und Zusammenarbeit gemeinsam erforderlich sind. Automatisierte Tests liefern frühe und wiederholbare Rückmeldungen; ihren tatsächlichen Nutzen erreichen sie aber erst durch klare Zuständigkeiten, risikobasierte Security Gates und eine fachliche Bewertung der Ergebnisse. Damit besteht die zentrale Verbesserung gegenüber einem reinen DevOps-Prozess nicht nur aus zusätzlichen Scannern, sondern aus einer durchgängigen Sicherheitsverantwortung von der Planung bis zum Monitoring.

Die Arbeit hat auch Grenzen. Es wurde keine eigene praktische Messung durchgeführt. Es wurde also nicht getestet, wie schnell oder genau einzelne Tools in einer echten Pipeline arbeiten. Die Bewertung basiert auf wissenschaftlicher und fachlicher Literatur. Für eine weiterführende Arbeit wäre es sinnvoll, eine Beispiel-Pipeline praktisch aufzubauen und zu untersuchen, wie sich SAST, DAST, SCA, Secret Scanning und Container Scanning konkret auf Laufzeit, Fehlalarme und Sicherheitsfunde auswirken.

Literatur- und Quellenverzeichnis

- Akbar, M. A.; Smolander, K.; Mahmood, S.; Alsanad, A. (2022): Toward successful DevSecOps in software development organizations: A decision-making framework. *Information and Software Technology*, 147, 106894. DOI: <https://doi.org/10.1016/j.infsof.2022.106894>
- Feio, C.; Santos, N.; Escravana, N.; Pacheco, B. (2024): An Empirical Study of DevSecOps Focused on Continuous Security Testing. 2024 IEEE European Symposium on Security and Privacy Workshops, S. 610-617. DOI: <https://doi.org/10.1109/EuroSPW61312.2024.00074>
- GitHub Docs (o. J.): About secret scanning. Abruf am 02.07.2026. <https://docs.github.com/code-security/secret-scanning/about-secret-scanning>
- Kumar, R.; Goyal, R. (2020): Modeling continuous security: A conceptual model for automated DevSecOps using open-source software over cloud (ADOC). *Computers & Security*, 97, 101967. DOI: <https://doi.org/10.1016/j.cose.2020.101967>
- Myrbakken, H.; Colomo-Palacios, R. (2017): DevSecOps: A Multivocal Literature Review. In: *Software Process Improvement and Capability Determination*, S. 17-29. DOI: https://doi.org/10.1007/978-3-319-67383-7_2
- OpenSSF/SLSA (o. J.): Supply-chain Levels for Software Artifacts. Abruf am 02.07.2026. <https://slsa.dev/>
- OWASP (o. J.-b): CI/CD Security Cheat Sheet. Abruf am 02.07.2026. https://cheatsheetseries.owasp.org/cheatsheets/CI_CD_Security_Cheat_Sheet.html
- OWASP (o. J.-c): OWASP Top 10 CI/CD Security Risks. Abruf am 02.07.2026. <https://owasp.org/www-project-top-10-ci-cd-security-risks/>
- OWASP (o. J.-d): DevSecOps Guideline. Abruf am 02.07.2026. <https://owasp.org/www-project-devsecops-guideline/>
- OWASP (o. J.-e): Dependency-Check. Abruf am 02.07.2026. <https://owasp.org/www-project-dependency-check/>
- OWASP (o. J.-j): Secrets Management Cheat Sheet. Abruf am 02.07.2026. https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
- OWASP (o. J.-k): Secure Code Review Cheat Sheet. Abruf am 02.07.2026. https://cheatsheetseries.owasp.org/cheatsheets/Secure_Code_Review_Cheat_Sheet.html
- OWASP (o. J.-l): Software Supply Chain Security Cheat Sheet. Abruf am 02.07.2026. https://cheatsheetseries.owasp.org/cheatsheets/Software_Supply_Chain_Security_Cheat_Sheet.html
- OWASP (o. J.-m): Static Code Analysis. Abruf am 02.07.2026. https://owasp.org/www-community/controls/Static_Code_Analysis
- OWASP ASVS (o. J.): Application Security Verification Standard. Abruf am 02.07.2026. <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Foundation (o. J.): About the OWASP Foundation. Abruf am 02.07.2026. <https://owasp.org/about/>
- OWASP SAMM (o. J.): Software Assurance Maturity Model. Abruf am 02.07.2026. <https://owasp.org/www-project-samm/>

- OWASP ZAP (o. J.): Zed Attack Proxy. Abruf am 02.07.2026. <https://www.zaproxy.org/>
- Prates, L.; Pereira, R. (2025): DevSecOps practices and tools. *International Journal of Information Security*, 24, Artikel 11. DOI: <https://doi.org/10.1007/s10207-024-00914-z>
- Rajapakse, R. N.; Zahedi, M.; Babar, M. A.; Shen, H. (2022a): Challenges and solutions when adopting DevSecOps: A systematic review. *Information and Software Technology*, 141, 106700. DOI: <https://doi.org/10.1016/j.infsof.2021.106700>
- Rajapakse, R. N.; Zahedi, M.; Babar, M. A. (2022b): Collaborative Application Security Testing for DevSecOps: An Empirical Analysis of Challenges, Best Practices and Tool Support. *Journal of Systems and Software*, 190, 111319. DOI: <https://doi.org/10.1016/j.jss.2022.111319>
- Souppaya, M.; Morello, J.; Scarfone, K. (2017): Application Container Security Guide. NIST Special Publication 800-190. <https://csrc.nist.gov/pubs/sp/800/190/final>
- Souppaya, M.; Scarfone, K.; Dodson, D. (2022): Secure Software Development Framework (SSDF) Version 1.1. NIST Special Publication 800-218. DOI: <https://doi.org/10.6028/NIST.SP.800-218>
- Zhao, X.; Clear, T.; Lal, R. (2024): Identifying the primary dimensions of DevSecOps: A multivocal literature review. *Journal of Systems and Software*, 214, 112063. DOI: <https://doi.org/10.1016/j.jss.2024.112063>

Eidesstattliche Erklärung

Hiermit erkläre ich an Eides Statt, dass ich die vorliegende Arbeit selbst angefertigt habe; die aus fremden Quellen direkt oder indirekt übernommenen Gedanken sind als solche kenntlich gemacht.

Die Arbeit wurde bisher keiner Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Ort, Datum: Bergheim, 07/07/2026

Unterschrift:  _____